TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

TRUNG TÂM PHÁT TRIỂN CÔNG NGHỆ THÔNG TIN

******

BÁO CÁO ĐỒ ÁN CUỐI KỲ

KỸ THUẬT LẬP TRÌNH PYTHON

Đề tài:

Hệ thống phân tích, xử lý cảnh báo từ Wazuh và gửi cảnh báo qua Telegram

Mã lớp học phần: IE221.F33.LT.CNTT
Giảng viên hướng dẫn: ThS. Nghi Hoàng Khoa

Nhóm sinh viên thực hiện:

STT	HỌ TÊN	MSSV
1	Trần Tấn Tài	25410296

Tp. Hồ Chí Minh, tháng 10 năm 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh, ngày....tháng.....năm 2026

Người nhận xét

(Ký tên ghi rõ họ tên)

MỤC LỤC

TÓM TẮT ĐỒ ÁN *5*
LỜI MỞ ĐẦU *6*
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI *7*
1.1 Lý do chọn đề tài **7**
1.2. Mục tiêu của đồ án **7**
1.3. Phạm vi của đồ án **7**
1.4. Đối tượng sử dụng **7**
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ *8*
2.1. Tổng quan về SIEM **8**
2.2. Nền tảng Wazuh **8**
2.3. Telegram Bot và Wazuh integration **9**
2.4. Ngôn ngữ Python **9**
2.5. Các công nghệ trong ứng dụng **9**
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG *10*
3.1. Phân tích yêu cầu **10**
3.2. Kiến trúc tổng thể **10**
3.3. Luồng gửi cảnh báo Telegram **11**
3.4. Luồng nhận log trực tiếp từ Wazuh của Desktop App **11**
3.5. Luồng nhập file JSON Desktop App **11**
3.6. Thiết kế cơ sở dữ liệu **12**
3.7. Thiết kế cấu trúc source code **12**
3.8. Thiết kế giao diện **12**
CHƯƠNG 4. PHÂN TÍCH CÁC TÍNH NĂNG CỦA CHƯƠNG TRÌNH *14*
4.1. Môi trường phát triển **14**
4.2. Xây dựng module Wazuh–Telegram **15**
4.3. Xây dựng integration gửi log đến project **16**
4.4. Xây dựng chức năng nhận log **16**
4.5. Kiểm tra và chuẩn hóa dữ liệu **16**
4.6. Lưu dữ liệu vào SQLite **17**
4.7. Xây dựng giao diện và biểu đồ **17**

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ 18
5.1. Mục tiêu kiểm thử 18
5.2. Các trường hợp kiểm thử 18
5.3. Kết quả kiểm thử 18
5.4. Đánh giá kết quả 19
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 19
1. Kết luận 19
2. Đánh giá việc áp dụng kiến thức Python 19
3. Hướng phát triển 19
TÀI LIỆU THAM KHẢO 20
PHỤ LỤC: KHAI BÁO SỬ DỤNG AI 20
1. Công cụ AI đã sử dụng 20
2. Mục đích sử dụng 20
3. Một số prompt tiêu biểu 20
4. Các file Markdown hỗ trợ 20

Mục lục bảng

Bảng 1: Danh sách tính năng 10
Bảng 2: Cấu trúc dữ liệu 12
Bảng 3: Môi trường phát triển 15
Bảng 4: Trường hợp kiểm thử 18

Mục lục hình ảnh

Hình 1: Dashboard tổng quan 13
Hình 2: Danh sách cảnh báo 14
Hình 3: Trạng thái kết nối 14

TÓM TẮT ĐỒ ÁN

Đồ án xây dựng hệ thống hỗ trợ cảnh báo và phân tích các sự kiện bảo mật được phát hiện bởi nền tảng SIEM Wazuh. Hệ thống gồm hai thành phần hoạt động độc lập: module tích hợp Wazuh với Telegram và ứng dụng Python desktop dùng để lưu trữ, phân tích dữ liệu cảnh báo.

Thành phần thứ nhất là module Wazuh–Telegram. Module này sử dụng cơ chế integration của Wazuh kết hợp với chương trình Python để đọc sự kiện, lựa chọn các trường thông tin cần thiết, định dạng nội dung và gửi cảnh báo đến Telegram Bot. Thành phần này hoạt động trực tiếp trên máy chủ Wazuh và không phụ thuộc vào ứng dụng phân tích biểu đồ.

Thành phần thứ hai là ứng dụng desktop được xây dựng bằng Python. Wazuh gửi raw JSON đến ứng dụng thông qua một integration riêng, sử dụng giao thức HTTP và API key để xác thực. Dữ liệu nhận được sẽ được kiểm tra, chuẩn hóa thành đối tượng Alert và lưu vào cơ sở dữ liệu SQLite. Ngoài ra, chương trình còn hỗ trợ nhập file JSON để kiểm thử và trình diễn khi không kết nối với Wazuh.

Ứng dụng sử dụng Tkinter để hiển thị danh sách cảnh báo, tìm kiếm, lọc dữ liệu, xem thông tin chi tiết và trình bày các biểu đồ thống kê theo mức độ, thời gian và agent. Người dùng có thể bật hoặc tắt chức năng nhận log Wazuh trực tiếp trên giao diện.

Đồ án vận dụng các kiến thức của môn Lập trình Python như kiểu dữ liệu, câu điều kiện, vòng lặp, hàm, module, xử lý ngoại lệ, lập trình hướng đối tượng và giao diện desktop. Kết quả của đồ án là một hệ thống gồm module gửi cảnh báo nhanh qua Telegram và ứng dụng phân tích trực quan dữ liệu bảo mật từ Wazuh.

LỜI MỞ ĐẦU

Trong quá trình chuyển đổi số, các hệ thống công nghệ thông tin ngày càng phải đối mặt với nhiều nguy cơ như truy cập trái phép, dò quét mật khẩu, mã độc và các hành vi bất thường. Việc theo dõi nhật ký thủ công thường mất nhiều thời gian và khó phát hiện kịp thời các sự kiện quan trọng. Vì vậy, những hệ thống SIEM có khả năng thu thập, phân tích và cảnh báo sự kiện bảo mật đóng vai trò cần thiết trong việc hỗ trợ người quản trị giám sát hệ thống.

Wazuh là một nền tảng SIEM mã nguồn mở, hỗ trợ thu thập log, phát hiện mối đe dọa và tạo cảnh báo theo các luật bảo mật. Tuy nhiên, dữ liệu cảnh báo của Wazuh thường có nhiều trường thông tin và cần được xử lý để người dùng có thể theo dõi thuận tiện hơn. Từ nhu cầu đó, nhóm lựa chọn đề tài **"Kết nối hệ thống SIEM phân tích hành vi và gửi cảnh báo qua Telegram"**.

Đồ án gồm hai thành phần hoạt động độc lập. Thành phần thứ nhất là module tích hợp giữa Wazuh và Telegram, sử dụng Python để xử lý sự kiện và gửi thông báo đến người quản trị. Thành phần thứ hai là ứng dụng Python desktop dùng để tiếp nhận raw JSON từ Wazuh, lưu dữ liệu vào SQLite và hiển thị kết quả dưới dạng bảng, thông tin chi tiết và biểu đồ thống kê. Chương trình cũng hỗ trợ nhập file JSON để thuận tiện cho việc kiểm thử và trình diễn.

Thông qua quá trình thực hiện đồ án, nhóm có cơ hội vận dụng các kiến thức đã học về biến và kiểu dữ liệu, câu điều kiện, vòng lặp, hàm, module, xử lý ngoại lệ, lập trình hướng đối tượng và xây dựng giao diện bằng Tkinter. Đồng thời, nhóm cũng hiểu rõ hơn về cách tổ chức một dự án Python, làm việc với JSON, SQLite, giao tiếp HTTP và quản lý mã nguồn bằng Git.

Nhóm xin chân thành cảm ơn giảng viên đã hướng dẫn và cung cấp kiến thức trong suốt quá trình học tập. Mặc dù đã cố gắng hoàn thiện đồ án, sản phẩm vẫn có thể còn một số hạn chế. Nhóm rất mong nhận được những ý kiến đóng góp từ giảng viên để tiếp tục cải thiện chương trình và nâng cao kiến thức trong thời gian tới.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

- Wazuh là nền tảng SIEM mã nguồn mở có khả năng thu thập, phân tích và phân loại các sự kiện bảo mật. Tuy nhiên, người quản trị cần được thông báo nhanh khi xuất hiện cảnh báo quan trọng, đồng thời cần một giao diện đơn giản để theo dõi và thống kê dữ liệu.
- Vì vậy, nhóm lựa chọn đề tài **"Kết nối hệ thống SIEM phân tích hành vi và gửi cảnh báo qua Telegram"**. Đồ án sử dụng module Wazuh–Telegram để gửi cảnh báo đến người quản trị và xây dựng một ứng dụng desktop độc lập để tiếp nhận, lưu trữ, tìm kiếm và trực quan hóa dữ liệu cảnh báo từ Wazuh. Đề tài có tính thực tế và phù hợp để vận dụng các kiến thức của môn Lập trình Python.

1.2. Mục tiêu của đồ án

Mục tiêu của đồ án là xây dựng hệ thống hỗ trợ cảnh báo và theo dõi các sự kiện bảo mật từ Wazuh. Hệ thống gồm hai thành phần hoạt động độc lập:

- Xây dựng module Wazuh–Telegram bằng Python để xử lý sự kiện và gửi cảnh báo đến người quản trị.
- Xây dựng ứng dụng desktop bằng Python để nhận raw JSON trực tiếp từ Wazuh.
- Hỗ trợ nhập dữ liệu từ file JSON để kiểm thử và trình diễn.
- Kiểm tra, chuẩn hóa và lưu cảnh báo vào cơ sở dữ liệu SQLite.
- Hiển thị danh sách, thông tin chi tiết và biểu đồ thống kê.
- Hỗ trợ tìm kiếm, lọc và bật hoặc tắt chức năng nhận log trên giao diện.

1.3. Phạm vi của đồ án

Đồ án sử dụng hệ thống Wazuh đã được cài đặt sẵn để thu thập và phân tích các sự kiện bảo mật. Nhóm không xây dựng lại nền tảng SIEM mà tập trung vào các chức năng sau:

- Xây dựng module Wazuh–Telegram để gửi cảnh báo theo mức độ đã cấu hình.
- Nhận cảnh báo Wazuh dạng raw JSON thông qua HTTP và xác thực bằng API key.
- Hỗ trợ nhập dữ liệu từ file JSON để kiểm thử và trình diễn.
- Kiểm tra, chuẩn hóa và lưu dữ liệu vào SQLite.
- Hiển thị danh sách, thông tin chi tiết và biểu đồ thống kê trên ứng dụng desktop.
- Tìm kiếm, lọc và ngăn lưu trùng cảnh báo dựa trên alert_id.

1.4. Đối tượng sử dụng

Đối tượng sử dụng của hệ thống là người quản trị hệ thống, nhân viên giám sát an toàn thông tin hoặc người đang học và nghiên cứu về Wazuh. Người dùng có thể nhận cảnh báo nhanh qua Telegram, theo dõi dữ liệu trên ứng dụng desktop, tìm kiếm sự kiện và xem các biểu đồ thống kê mà không cần đọc trực tiếp toàn bộ raw log của Wazuh.

1.5. Phương pháp thực hiện

Đồ án được thực hiện trên hệ thống Wazuh đã được cài đặt và hoạt động ổn định. Người thực hiện không cài đặt lại Wazuh mà tập trung xây dựng các thành phần tích hợp và ứng dụng phân tích dữ liệu.
Quá trình thực hiện gồm:

- Xây dựng integration riêng để chuyển raw JSON từ Wazuh đến ứng dụng desktop.

- Xây dựng các lớp và hàm kiểm tra, chuẩn hóa dữ liệu.
- Sử dụng SQLite để lưu trữ và chống trùng cảnh báo, thiết kế giao diện Tkinter và biểu đồ bằng Matplotlib.
- Kiểm thử bằng dữ liệu JSON mẫu và cảnh báo thực tế từ Wazuh.
- Quản lý source code và lịch sử phát triển bằng GitHub.

1.6. Kết quả đạt được

Sau quá trình thực hiện, đồ án đã hoàn thành các chức năng chính:

- Module Wazuh–Telegram gửi cảnh báo đến người quản trị,
- Ứng dụng desktop nhận raw JSON trực tiếp từ Wazuh qua HTTP và API key.
- Kiểm tra, chuẩn hóa và chuyển dữ liệu thành đối tượng Alert. lưu dữ liệu vào SQLite
- Hiển thị danh sách, tìm kiếm, lọc và xem chi tiết cảnh báo.
- Thống kê dữ liệu bằng biểu đồ theo mức độ, thời gian và agent.
- Cho phép bật hoặc tắt chức năng nhận log trên giao diện.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Tổng quan về SIEM

SIEM (Security Information and Event Management) là hệ thống thu thập, lưu trữ và phân tích dữ liệu bảo mật từ nhiều nguồn như máy chủ, thiết bị mạng, ứng dụng và thiết bị đầu cuối. SIEM giúp tập trung dữ liệu log tại một nơi để người quản trị dễ dàng theo dõi và phát hiện các dấu hiệu bất thường.

2.2. Nền tảng Wazuh

Wazuh là một nền tảng SIEM mã nguồn mở, hỗ trợ giám sát an toàn thông tin và phát hiện các mối đe dọa. Hệ thống có thể thu thập log từ các agent, phân tích dữ liệu và tạo cảnh báo khi phát hiện sự kiện phù hợp với các luật bảo mật.

2.2.1. Thu thập và phân tích log

Wazuh Agent được cài đặt trên thiết bị cần giám sát để thu thập log và gửi về Wazuh Manager. Manager tiếp nhận dữ liệu, phân tích nội dung và xác định sự kiện có dấu hiệu bất thường hay không.

2.2.2. Rule và mức độ cảnh báo

Wazuh sử dụng các rule để nhận diện và phân loại sự kiện bảo mật. Mỗi cảnh báo có một rule_id, nội dung mô tả và mức độ rule_level.

Mức cảnh báo càng cao thì sự kiện càng cần được chú ý. Đồ án sử dụng điều kiện level trong block integration để xác định những cảnh báo được gửi đến Telegram hoặc ứng dụng desktop.

2.2.3. Dữ liệu cảnh báo JSON

Cảnh báo Wazuh có thể được biểu diễn dưới dạng JSON. Các trường chính được sử dụng trong đồ án gồm:

- id: mã cảnh báo.
- timestamp: thời gian phát sinh.
- agent: thông tin thiết bị gửi log.

- rule: mã rule, level và mô tả.
- groups: nhóm sự kiện.
- location: nguồn phát sinh log.

JSON có cấu trúc rõ ràng, dễ truyền qua HTTP và thuận tiện để Python chuyển đổi thành dictionary trước khi xử lý.

2.2.4. Cơ chế custom integration

Custom integration cho phép Wazuh gọi một chương trình bên ngoài khi cảnh báo thỏa mãn điều kiện đã cấu hình trong ossec.conf.

Cơ chế này giúp mở rộng chức năng của Wazuh mà không cần thay đổi quá trình phân tích log bên trong hệ thống.

2.3. Telegram Bot và Wazuh integration

Telegram Bot là công cụ cho phép chương trình tự động gửi tin nhắn đến người dùng hoặc nhóm Telegram thông qua Telegram Bot API. Trong đồ án, Telegram được sử dụng để thông báo nhanh các cảnh báo bảo mật quan trọng do Wazuh phát hiện.

Module Wazuh–Telegram hoạt động độc lập với ứng dụng.

2.4. Ngôn ngữ Python

Python được lựa chọn vì có cú pháp rõ ràng, dễ đọc và phù hợp với nội dung môn học. Ngôn ngữ này cung cấp các thư viện hỗ trợ xử lý JSON, giao tiếp HTTP, làm việc với SQLite, xây dựng giao diện desktop và hiển thị biểu đồ.

2.5. Các công nghệ trong ứng dụng

2.5.1. Tkinter

Tkinter là thư viện giao diện có sẵn trong Python. Đồ án sử dụng Tkinter để xây dựng cửa sổ desktop, các nút chức năng, bảng dữ liệu, bộ lọc và khu vực hiển thị trạng thái hệ thống.

2.5.2. SQLite

SQLite là cơ sở dữ liệu dạng file, không yêu cầu cài đặt máy chủ riêng. Dữ liệu cảnh báo được lưu trong file alerts.db, bảng alerts. Trường alert_id được sử dụng làm khóa chính để hạn chế lưu trùng dữ liệu.

2.5.3. Matplotlib

Matplotlib được sử dụng để tạo biểu đồ và nhúng trực tiếp vào giao diện Tkinter. Ứng dụng hiển thị ba biểu đồ chính: số cảnh báo theo mức độ, số cảnh báo theo ngày và các agent có nhiều cảnh báo.

2.5.4. JSON

JSON là định dạng dữ liệu chính của cảnh báo Wazuh. Python chuyển dữ liệu JSON thành dictionary để kiểm tra, chuẩn hóa và tạo đối tượng Alert. Đồ án hỗ trợ cả dữ liệu JSON nhận trực tiếp từ Wazuh và file JSON do người dùng nhập.

2.5.5. HTTP và API key

Wazuh gửi raw JSON đến ứng dụng thông qua HTTP. WazuhReceiver mở cổng nhận dữ liệu và kiểm tra API key trong header X-API-Key. Chỉ yêu cầu có khóa hợp lệ mới được tiếp nhận, giúp hạn chế dữ liệu gửi từ nguồn không xác định.

2.5.6. Git và GitHub

Git được sử dụng để theo dõi lịch sử thay đổi của source code. GitHub lưu trữ repository, tài liệu và dữ liệu mẫu, giúp dễ dàng sao lưu, tải dự án sang máy khác và theo dõi quá trình phát triển. Các file chứa thông tin bí mật như API key, Telegram token và database phát sinh không được đưa lên GitHub.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

Hệ thống được thiết kế gồm hai thành phần độc lập: module Wazuh–Telegram và ứng dụng desktop phân tích cảnh báo. Hai thành phần cùng sử dụng dữ liệu từ Wazuh nhưng không gọi trực tiếp lẫn nhau.

Yêu cầu chức năng

Chức năng	Mô tả
Gửi cảnh báo qua Telegram	Module Wazuh–Telegram nhận sự kiện, lựa chọn thông tin cần thiết và gửi đến Telegram Bot.
Nhận log trực tiếp từ Wazuh	Ứng dụng mở HTTP receiver để nhận raw JSON do integration của Wazuh gửi đến.
Xác thực nguồn gửi	Receiver kiểm tra API key trong header trước khi chấp nhận dữ liệu.
Nhập file JSON	Người dùng chọn file cảnh báo Wazuh có sẵn để nhập dữ liệu thủ công.
Kiểm tra dữ liệu	Chương trình kiểm tra cấu trúc và các trường bắt buộc trước khi lưu.
Lưu dữ liệu SQLite	Cảnh báo hợp lệ được lưu vào bảng alerts trong file alerts.db.
Chống trùng dữ liệu	Cảnh báo có alert_id đã tồn tại sẽ không được thêm lại.
Tìm kiếm và lọc	Cho phép tìm theo từ khóa, mức độ và tên agent.
Xem chi tiết	Hiển thị đầy đủ thông tin và raw JSON của cảnh báo được chọn.
Hiển thị biểu đồ	Thống kê cảnh báo theo mức độ, thời gian và agent.
Bật hoặc tắt receiver	Người dùng điều khiển trạng thái nhận log trực tiếp trên giao diện.

Bảng 1: Danh sách tính năng

3.2. Kiến trúc tổng thể

Hệ thống gồm các thành phần chính sau:

1. **Wazuh Agent:** Thu thập log từ thiết bị được giám sát và gửi về Wazuh Manager.
2. **Wazuh Manager:** Phân tích log bằng các rule, xác định mức độ và tạo cảnh báo JSON.
3. **Module Wazuh–Telegram:** Nhận sự kiện từ Wazuh, định dạng các trường cần thiết và gửi thông báo đến Telegram Bot. Module này hoạt động độc lập với ứng dụng desktop.
4. **Integration của project:** Nhận file cảnh báo từ Wazuh và chuyển raw JSON đến địa chỉ HTTP của ứng dụng.
5. **HTTP Receiver:** Mở cổng nhận dữ liệu, kiểm tra API key và đưa cảnh báo hợp lệ vào hàng đợi xử lý.

6. **Đối tượng Alert:** Chuẩn hóa dữ liệu JSON thành cấu trúc thống nhất để chương trình sử dụng.
7. **Cơ sở dữ liệu SQLite:** Lưu cảnh báo vào bảng alerts và ngăn dữ liệu trùng bằng alert_id.
8. **Dịch vụ phân tích:** Đếm và nhóm dữ liệu theo mức độ, thời gian và agent để phục vụ biểu đồ.
9. **Giao diện desktop:** Hiển thị danh sách, thông tin chi tiết, bộ lọc, biểu đồ và trạng thái receiver.
10. **File JSON:** Là nguồn dữ liệu thay thế, được sử dụng khi kiểm thử hoặc trình diễn mà không cần kết nối Wazuh.

3.3. Luồng gửi cảnh báo Telegram

Luồng gửi cảnh báo Telegram được thực hiện như sau:

1. Wazuh Agent thu thập log và gửi dữ liệu về Wazuh Manager.
2. Wazuh Manager phân tích sự kiện bằng các rule bảo mật.
3. Khi mức cảnh báo thỏa mãn điều kiện trong ossec.conf, Wazuh gọi integration custom-telegramalert-level.
4. File custom-telegramalert-level khởi chạy chương trình custom-telegramalert-level.py.
5. Chương trình Python đọc file JSON của cảnh báo do Wazuh cung cấp.
6. Các trường cần thiết như thời gian, agent, địa chỉ IP, rule, level và mô tả được lấy ra để tạo nội dung thông báo.
7. Chương trình sử dụng requests.post() để gửi nội dung đến Telegram Bot API.
8. Telegram Bot chuyển thông báo đến người dùng hoặc nhóm đã được cấu hình.

3.4. Luồng nhận log trực tiếp từ Wazuh của Desktop App

Chức năng nhận log trực tiếp sử dụng một integration riêng, không phụ thuộc vào module Telegram. Quá trình xử lý gồm:

1. Wazuh phát hiện cảnh báo thỏa mãn mức level đã cấu hình.
2. Wazuh gọi file custom-project-alerts.
3. File này khởi chạy custom-project-alerts.py.
4. Chương trình Python đọc raw JSON và gửi yêu cầu HTTP POST đến endpoint /alerts.
5. API key được đặt trong header X-API-Key.
6. WazuhReceiver tiếp nhận yêu cầu và so sánh API key với giá trị trong project_config.conf.
7. Nếu API key sai, receiver trả về mã 401 Unauthorized.
8. Nếu API key đúng, receiver kiểm tra kích thước và định dạng JSON.
9. Raw JSON hợp lệ được đưa vào hàng đợi để không làm treo giao diện.
10. Giao diện kiểm tra hàng đợi định kỳ và gọi Alert.from_wazuh_dict() để chuẩn hóa dữ liệu.
11. Đối tượng Alert được lưu vào bảng alerts trong SQLite.
12. Danh sách và biểu đồ được cập nhật khi có cảnh báo mới.

3.5. Luồng nhập file JSON Desktop App

Chức năng nhập file JSON được sử dụng để kiểm thử hoặc trình diễn khi không nhận dữ liệu trực tiếp từ Wazuh.

Quá trình xử lý gồm:

1. Người dùng nhấn nút **Nhập file JSON** trên giao diện.
2. Hàm import_json() mở hộp thoại để người dùng chọn file.
3. AlertReader.read_file() đọc toàn bộ nội dung JSON.

4. Hàm _parse_records() xác định và tách các bản ghi trong file.
5. Từng bản ghi được kiểm tra kiểu dữ liệu và các trường bắt buộc.
6. Alert.from_wazuh_dict() chuẩn hóa bản ghi thành đối tượng Alert.
7. Các cảnh báo hợp lệ được đưa vào danh sách, còn bản ghi lỗi được ghi nhận riêng.
8. DatabaseService.save_alerts() lưu danh sách cảnh báo vào SQLite.
9. Những alert_id đã tồn tại sẽ được bỏ qua.
10. Giao diện hiển thị số bản ghi mới, bản ghi trùng và bản ghi lỗi.
11. Bảng dữ liệu và biểu đồ được cập nhật từ SQLite.

3.6. Thiết kế cơ sở dữ liệu

Đồ án sử dụng SQLite và lưu dữ liệu trong file data/alerts.db. Database hiện có một bảng alerts; mỗi dòng tương ứng với một cảnh báo Wazuh.

Cột	Kiểu dữ liệu	Ý nghĩa
alert_id	TEXT	Mã cảnh báo và khóa chính
timestamp	TEXT	Thời gian phát sinh cảnh báo
agent_id	TEXT	Mã thiết bị Wazuh Agent
agent_name	TEXT	Tên thiết bị
agent_ip	TEXT	Địa chỉ IP của thiết bị
rule_id	TEXT	Mã rule Wazuh
rule_level	INTEGER	Mức độ cảnh báo
description	TEXT	Nội dung mô tả
groups_json	TEXT	Danh sách nhóm sự kiện
location	TEXT	Nguồn phát sinh log
telegram_sent	INTEGER	Trạng thái gửi Telegram được giữ trong cấu trúc dữ liệu
raw_json	TEXT	Dữ liệu JSON gốc

Bảng 2: Cấu trúc dữ liệu

3.7. Thiết kế cấu trúc source code

Source code được chia theo từng nhóm chức năng.
Vai trò của các thành phần chính:

- main.py: điểm bắt đầu của chương trình, khởi tạo database và giao diện.
- integration/: chứa các file tích hợp Wazuh–Telegram và Wazuh–Project.
- alerts_process.py: mở HTTP receiver và tiếp nhận raw JSON.
- alert.py: định nghĩa lớp Alert và chuẩn hóa dữ liệu Wazuh.
- alert_reader.py: đọc và kiểm tra dữ liệu từ file JSON.
- database_service.py: khởi tạo và thao tác với SQLite.
- analysis_service.py: thống kê dữ liệu phục vụ biểu đồ.
- main_window.py: xây dựng giao diện và điều khiển các chức năng.
- data/: chứa dữ liệu mẫu và database được tạo khi chạy.

3.8. Thiết kế giao diện

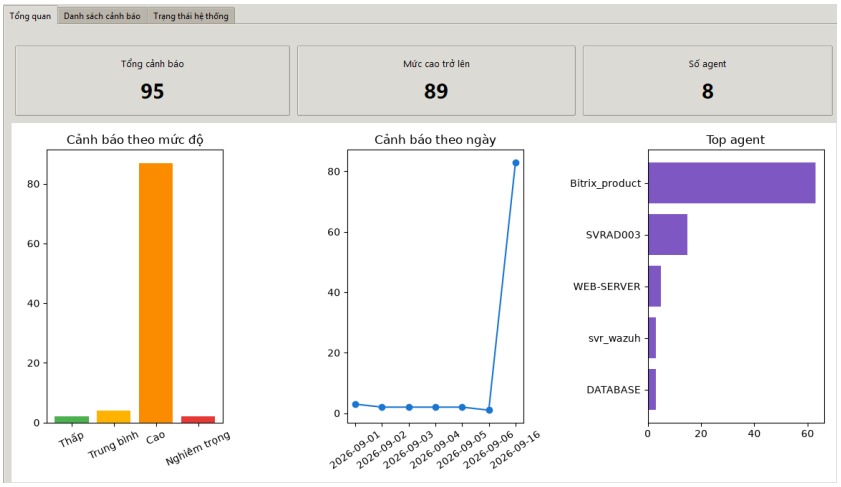
Giao diện được xây dựng bằng Tkinter theo hướng đơn giản, tập trung vào các chức năng cần thiết. Cửa sổ chính gồm thanh công cụ và ba trang chức năng.

3.8.1. Thanh công cụ

Thanh công cụ cung cấp hai chức năng:

- **Nhập file JSON:** chọn file cảnh báo để đưa vào hệ thống.
- **Làm mới:** đọc lại dữ liệu từ SQLite và cập nhật giao diện.

3.8.2. Trang Tổng quan



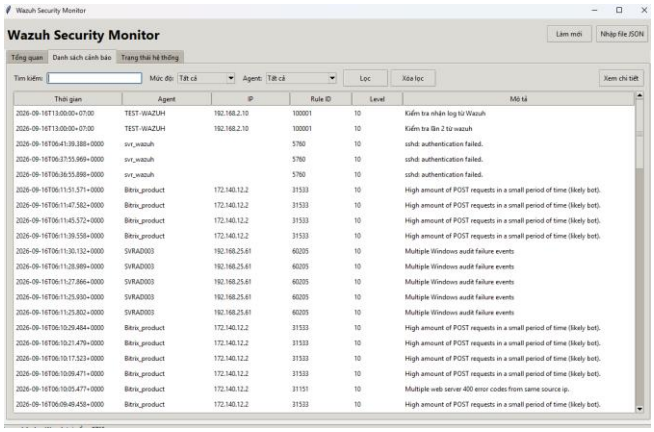
Hình 1: Dashboard tổng quan

Trang Tổng quan hiển thị các thông tin thống kê chính:

- Tổng số cảnh báo.
- Số cảnh báo từ mức cao trở lên.
- Số agent xuất hiện trong dữ liệu.
- Biểu đồ số cảnh báo theo mức độ.
- Biểu đồ số cảnh báo theo ngày.
- Biểu đồ các agent có nhiều cảnh báo.

Trang này giúp người dùng nhanh chóng đánh giá tình trạng cảnh báo mà không cần đọc từng dòng dữ liệu.

3.8.3. Trang Danh sách cảnh báo

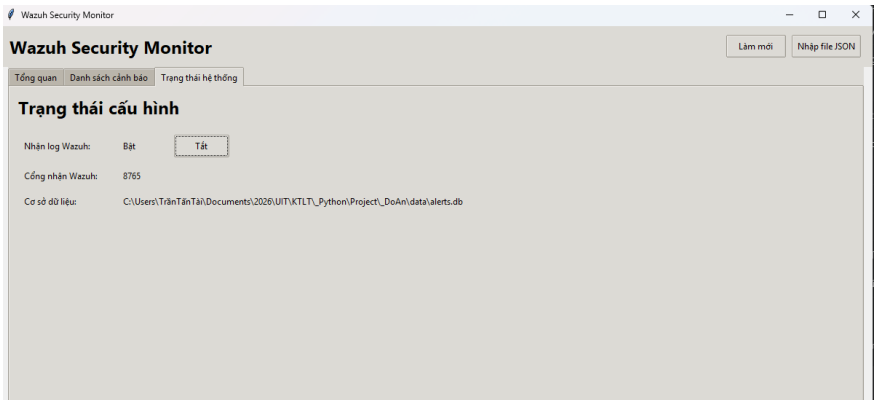


Hình 2: Danh sách cảnh báo

Dữ liệu được trình bày dưới dạng bảng với các trường như thời gian, agent, địa chỉ IP, rule ID, level và mô tả. Người dùng có thể:

- Tìm kiếm bằng từ khóa.
- Lọc theo mức độ cảnh báo.
- Lọc theo tên agent.
- Xóa điều kiện lọc.
- Xem thông tin chi tiết và raw JSON của cảnh báo.

3.8.4. Trang Trạng thái hệ thống



Hình 3: Trạng thái kết nối

Trang này hiển thị:

- Trạng thái nhận log Wazuh.
- Nút bật hoặc tắt HTTP receiver.
- Cổng đang sử dụng để nhận log.
- Đường dẫn đến file cơ sở dữ liệu.

CHƯƠNG 4. PHÂN TÍCH CÁC TÍNH NĂNG CỦA CHƯƠNG TRÌNH

4.1. Môi trường phát triển

Đồ án được phát triển và kiểm thử trên hai môi trường: máy chủ Wazuh và máy tính chạy ứng dụng desktop.

Thành phần	Công nghệ sử dụng
Máy chủ SIEM	Wazuh Manager đã cài đặt trên Linux
Máy chạy ứng dụng	Hệ điều hành Windows
Ngôn ngữ	Python 3.14
Công cụ lập trình	Visual Studio Code
Quản lý source code	Git và GitHub
Giao diện	Tkinter
Cơ sở dữ liệu	SQLite
Biểu đồ	Matplotlib
Giao tiếp HTTP	http.server và Requests
Định dạng dữ liệu	JSON
Kiểm thử	Unittest

Bảng 3: Môi trường phát triển

4.2. Xây dựng module Wazuh–Telegram

Module Wazuh–Telegram được triển khai trên máy chủ Wazuh và hoạt động độc lập với ứng dụng desktop. Module gồm block cấu hình trong ossec.conf và hai file custom-telegramalert-level, custom-telegramalert-level.py.

4.2.1. Cấu hình integration

Block integration quy định tên chương trình, mức cảnh báo và thông tin Telegram:

```
<integration>
  <name>custom-telegramalert-level</name>
  <level>10</level>
  <api_key>TELEGRAM_CHAT_ID</api_key>
  <hook_url>
    https://api.telegram.org/botBOT_TOKEN/sendMessage
  </hook_url>
  <alert_format>json</alert_format>
</integration>
```

Trong đó:

- name: tên file integration được Wazuh gọi.
- level: chỉ xử lý cảnh báo từ mức cấu hình trở lên.
- api_key: thông tin xác định nơi nhận tin nhắn.
- hook_url: địa chỉ Telegram Bot API.
- alert_format: yêu cầu Wazuh cung cấp cảnh báo dạng JSON.

4.2.2. File khởi chạy integration

File custom-telegramalert-level là shell script được Wazuh gọi khi sự kiện thỏa mãn điều kiện. File này xác định đường dẫn Wazuh và khởi chạy chương trình:

custom-telegramalert-level.py

Việc sử dụng file khởi chạy giúp chương trình Python hoạt động đúng với cơ chế custom integration của Wazuh.

4.2.3. Xử lý và gửi cảnh báo

File custom-telegramalert-level.py thực hiện các công việc:

1. Nhận đường dẫn file cảnh báo và thông tin cấu hình từ Wazuh.
2. Đọc nội dung cảnh báo JSON.
3. Lấy các trường cần thiết như thời gian, agent, IP, rule, level và mô tả.
4. Định dạng dữ liệu thành nội dung dễ đọc.
5. Gửi tin nhắn đến Telegram Bot API.

Đoạn lệnh chính dùng để gửi thông báo:

requests.post(hook_url, headers=headers,data=json.dumps(msg_data))

4.3. Xây dựng integration gửi log đến project

Integration này được cài đặt trên máy chủ Wazuh để chuyển raw JSON đến ứng dụng desktop. Thành phần này hoạt động độc lập với integration Telegram.

- **custom-project-alerts:** Là file shell được Wazuh gọi khi cảnh báo thỏa mãn mức level trong ossec.conf. File này xác định môi trường Wazuh và khởi chạy custom-project-alerts.py.
- **custom-project-alerts.py:** Đọc file cảnh báo JSON do Wazuh cung cấp, sau đó gửi toàn bộ dữ liệu đến ứng dụng bằng HTTP POST. Hàm main() chịu trách nhiệm nhận tham số, đọc JSON, tạo HTTP header và thực hiện yêu cầu gửi dữ liệu.
- **URL của receiver:** Có dạng http://IP_MAY_PROJECT:8765/alerts. Địa chỉ IP là máy chạy ứng dụng, cổng 8765 là cổng tiếp nhận và /alerts là endpoint nhận cảnh báo.
- **API key:** Được Wazuh truyền cho chương trình và đặt trong header X-API-Key. Giá trị này phải giống với API key trong project_config.conf của ứng dụng.

4.4. Xây dựng chức năng nhận log

Chức năng nhận log được xây dựng trong file integration/alerts_process.py, với thành phần chính là lớp WazuhReceiver.

Hàm load_receiver_config() đọc các thông tin trong project_config.conf, gồm trạng thái hoạt động, địa chỉ host, cổng, API key và kích thước dữ liệu tối đa.

Lớp WazuhReceiver gồm các hàm chính:

- start(): tạo ThreadingHTTPServer, mở endpoint /alerts và chạy receiver trên một luồng riêng.
- get_pending(): lấy các raw JSON đang chờ trong hàng đợi.
- stop(): đóng HTTP server khi người dùng tắt receiver hoặc đóng chương trình.
- is_running: kiểm tra receiver có thực sự hoạt động hay không.

Khi nhận yêu cầu HTTP, receiver kiểm tra:

- Đường dẫn phải là /alerts.
- API key trong X-API-Key phải chính xác.
- Kích thước dữ liệu phải nằm trong giới hạn.
- Nội dung phải là một JSON object hợp lệ.

4.5. Kiểm tra và chuẩn hóa dữ liệu

Việc chuẩn hóa dữ liệu được thực hiện bởi phương thức Alert.from_wazuh_dict() trong file src/models/alert.py.

Phương thức này nhận một dictionary và lấy các trường cần thiết từ cấu trúc cảnh báo Wazuh, gồm:

- Thời gian phát sinh.
- Thông tin agent.
- Rule ID và rule level.
- Nội dung mô tả.
- Nhóm sự kiện.
- Nguồn phát sinh log.

Một cảnh báo được xem là hợp lệ khi:

- Dữ liệu đầu vào là dictionary.
- Có trường timestamp.
- rule_level có thể chuyển thành số nguyên.

4.6. Lưu dữ liệu vào SQLite

Chức năng lưu trữ được xây dựng trong lớp DatabaseService tại file src/services/database_service.py.

Các hàm chính gồm:

- connect(): tạo kết nối đến file data/alerts.db.
- initialize(): tạo bảng alerts nếu bảng chưa tồn tại.
- save_alerts(): lưu danh sách đối tượng Alert vào database.
- get_alerts(): đọc dữ liệu và áp dụng điều kiện tìm kiếm, lọc.
- get_agent_names(): lấy danh sách tên agent phục vụ bộ lọc.

4.7. Xây dựng giao diện và biểu đồ

Giao diện được xây dựng trong file src/ui/main_window.py bằng lớp MainWindow. Chương trình sử dụng Tkinter cho các thành phần giao diện và Matplotlib để hiển thị biểu đồ.

4.7.1 Hiển thị danh sách cảnh báo

Hàm _create_alert_table() tạo bảng ttk.Treeview. Mỗi dòng hiển thị thời gian, agent, địa chỉ IP, rule ID, level và mô tả. Hàm _update_table() đọc danh sách đối tượng Alert và cập nhật các dòng trong bảng.

4.7.2 Tìm kiếm và lọc dữ liệu

Người dùng có thể tìm kiếm theo từ khóa và lọc theo mức độ hoặc tên agent. Hàm apply_filters() lấy điều kiện trên giao diện và gọi DatabaseService.get_alerts() để truy vấn dữ liệu phù hợp. Hàm clear_filters() xóa các điều kiện và hiển thị lại toàn bộ cảnh báo.

4.7.3 Hiển thị thông tin chi tiết

Hàm show_detail() lấy cảnh báo đang được chọn và mở một cửa sổ mới. Cửa sổ này hiển thị các trường đã chuẩn hóa và toàn bộ raw JSON, giúp người dùng kiểm tra dữ liệu gốc khi cần thiết.

4.7.4 Hiển thị biểu đồ

Hàm _update_charts() sử dụng kết quả từ AnalysisService để tạo ba biểu đồ:

- Biểu đồ cột thống kê cảnh báo theo mức độ.
- Biểu đồ đường thống kê số cảnh báo theo ngày.
- Biểu đồ thanh ngang hiển thị các agent có nhiều cảnh báo.

Các biểu đồ Matplotlib được nhúng vào Tkinter bằng FigureCanvasTkAgg và tự động cập nhật khi dữ liệu thay đổi.

4.7.5 Nút bật hoặc tắt nhận log

Trang trạng thái hệ thống cung cấp nút điều khiển HTTP receiver. Khi bật, chương trình mở cổng nhận log Wazuh; khi tắt, server được đóng và không tiếp nhận yêu cầu mới. Trạng thái chỉ hiển thị Bật khi receiver thực sự mở cổng thành công.

Hàm refresh_data() được sử dụng để đọc lại dữ liệu từ SQLite, cập nhật các thẻ thống kê, bảng cảnh báo và toàn bộ biểu đồ trên giao diện.

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Mục tiêu kiểm thử

Kiểm thử được thực hiện nhằm xác định các chức năng chính có hoạt động đúng với yêu cầu hay không. Nội dung kiểm thử tập trung vào:

- Khả năng đọc và xử lý dữ liệu JSON.
- Phát hiện dữ liệu thiếu trường bắt buộc.
- Ngăn lưu trùng cảnh báo trong SQLite.
- Xác thực API key khi nhận log qua HTTP.
- Khả năng xử lý lỗi mà không làm dừng chương trình.

5.2. Các trường hợp kiểm thử

Mã	Trường hợp	Dữ liệu kiểm thử	Kết quả mong đợi	Kết quả
TC01	Nhập file JSON hợp lệ	File sample_alerts.json đúng cấu trúc	Đọc được dữ liệu, tạo các đối tượng Alert và lưu vào SQLite	Đạt
TC02	Dữ liệu thiếu trường	Cảnh báo không có timestamp	Ghi nhận lỗi và không tạo đối tượng Alert	Đạt
TC03	Nhập cảnh báo trùng	Hai cảnh báo có cùng alert_id	Chỉ lưu một dòng, cảnh báo thứ hai được tính là trùng	Đạt
TC04	Gửi HTTP với API key đúng	JSON hợp lệ và API key trùng cấu hình	Receiver trả về 202 Accepted và đưa cảnh báo vào hàng đợi xử lý	Đạt

Bảng 4: Trường hợp kiểm thử

5.3. Kết quả kiểm thử

Toàn bộ **10/10 bài kiểm thử tự động đều thành công**. Các nội dung được kiểm tra gồm đọc JSON, phát hiện dữ liệu thiếu, thống kê, lọc dữ liệu, lưu SQLite, chống trùng, điều kiện gửi Telegram và xác thực HTTP receiver.

Kiểm thử thủ công cũng đạt các kết quả sau:

- Module Wazuh–Telegram gửi được cảnh báo đến Telegram Bot.
- Máy chủ Wazuh kết nối thành công đến cổng 8765 của ứng dụng.
- API key đúng nhận phản hồi 202 Accepted.
- API key sai nhận phản hồi 401 Unauthorized.
- Raw JSON từ Wazuh được chuẩn hóa và lưu vào SQLite.
- Cảnh báo trùng alert_id không được lưu lại.
- Dữ liệu mới xuất hiện trên bảng và biểu đồ.
- Chức năng tìm kiếm và lọc trả về dữ liệu phù hợp.
- Nút bật hoặc tắt receiver hoạt động đúng.

5.4. Đánh giá kết quả

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đồ án đã xây dựng được hệ thống hỗ trợ theo dõi cảnh báo bảo mật từ Wazuh với hai thành phần độc lập. Module Wazuh–Telegram có khả năng xử lý sự kiện và gửi thông báo đến người quản trị. Ứng dụng desktop có khả năng nhận raw JSON trực tiếp từ Wazuh, nhập dữ liệu từ file JSON, chuẩn hóa thành đối tượng Alert và lưu vào SQLite.

Ứng dụng cung cấp các chức năng xem danh sách, tìm kiếm, lọc, xem chi tiết và hiển thị biểu đồ theo mức độ, thời gian và agent. Chức năng bật hoặc tắt receiver giúp người dùng điều khiển việc nhận log trực tiếp trên giao diện. Các chức năng chính đã được kiểm thử tự động và thủ công với kết quả đạt yêu cầu.

2. Đánh giá

Quá trình thực hiện đồ án đã vận dụng các nội dung chính:

- Kiểu dữ liệu, câu điều kiện và vòng lặp để xử lý cảnh báo.
- Xử lý ngoại lệ đối với file, JSON, database và kết nối HTTP.
- Lập trình hướng đối tượng thông qua các lớp Alert, DatabaseService và WazuhReceiver.
- Tkinter để xây dựng giao diện desktop, SQLite để lưu trữ dữ liệu, Matplotlib để trực quan hóa kết quả.
- Thread và queue để nhận log mà không làm treo giao diện.

3. Hướng phát triển

Trong tương lai, chương trình có thể được mở rộng theo các hướng:

- Xuất báo cáo cảnh báo ra Excel, PDF hoặc Word.
- Sử dụng HTTPS để bảo vệ dữ liệu khi truyền qua mạng.
- Mã hóa API key và các thông tin cấu hình quan trọng.
- Triển khai HTTP receiver thành dịch vụ chạy nền.
- Bổ sung thêm biểu đồ và khả năng phân tích dữ liệu lớn.

- Nghiên cứu áp dụng học máy để phát hiện hoặc dự đoán hành vi bất thường.

TÀI LIỆU THAM KHẢO

1. Python Software Foundation, *sqlite3 — DB-API 2.0 interface for SQLite databases*.: https://docs.python.org/3/library/sqlite3.html
2. Wazuh, *External API integration and custom integration*: https://documentation.wazuh.com/current/user-manual/manager/integration-with-external-apis.html
3. SQLite, *SQLite Documentation*. https://www.sqlite.org/docs.html
4. Matplotlib, *Embedding, Matplotlib in Tkinter:* .https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_tk_sgskip.html
5. Telegram, *Telegram Bot API*. https://core.telegram.org/bots/api

PHỤ LỤC: KHAI BÁO SỬ DỤNG AI

1. Công cụ AI đã sử dụng

Sử dụng công cụ AI của OpenAI trong quá trình xây dựng và hoàn thiện đồ án.

2. Mục đích sử dụng

AI được sử dụng để:

- Gợi ý cấu trúc thư mục và cách chia module Python.
- Giải thích các hàm và đoạn code quan trọng.
- Hỗ trợ phát hiện lỗi và xây dựng trường hợp kiểm thử.
- Gợi ý nội dung README, báo cáo, slide và kịch bản demo.

3. Một số prompt tiêu biểu

- "Hãy phân tích luồng nhập file JSON và lưu dữ liệu vào SQLite."
- "Giải thích chức năng của Alert.from_wazuh_dict() bằng từ ngữ đơn giản."
- "Xây dựng HTTP receiver nhận raw JSON từ Wazuh và xác thực bằng API key."
- "Tạo giao diện Tkinter để hiển thị danh sách và biểu đồ cảnh báo."
- "Viết các trường hợp kiểm thử cho SQLite và Wazuh receiver."
- "Tóm tắt chức năng của từng file để phục vụ báo cáo đồ án."

4. Các file Markdown hỗ trợ

Các tài liệu Markdown được sử dụng để theo dõi và mô tả đồ án gồm:

- README.md: hướng dẫn cài đặt và chạy chương trình.
- docs/01_KE_HOACH_VA_LUONG_HE_THONG.md: kế hoạch và luồng hoạt động.
- docs/02_DAC_TA_TINH_NANG.md: đặc tả các chức năng.
- docs/03_GIAI_THICH_CODE_QUAN_TRONG.md: giải thích các hàm và biến quan trọng.
- deliverables/demo/KICH_BAN_DEMO.md: kịch bản thực hiện video demo.